

Zero-Dependency Browser OMI

A First-Principles Introduction to Math, Framing, and Agreement

Brian Thorne

Independent Researcher — Decentralized Topological Consensus & Autonomous AI
Universal Life Protocol
Los Angeles, CA
bthornemail@gmail.com
github.com/bthornemail
linkedin.com/in/brian-thorne-5b8a96112

1. The Starting Point

The Omi Object Model begins with a simple question:

What is the smallest thing two observers can agree on?

Not a server.

Not a blockchain.

Not a virtual machine.

Not a container.

Not a socket.

Not a database.

Not a barcode.

Not a font engine.

Not even a network.

The smallest thing is a notation.

```
omi---imo
```

That notation says:

```
source opens  
relation bridges  
object closes
```

Or, in the human reading:

```
On My Intelligence --- In My Opinion
```

This captures the basic human problem of meaning:

```
what I know internally  
must become externally inspectable  
before it can become shared agreement
```

OMI is the attempt to make that bridge exact.

2. The Browser Was Already Enough

The first major realization is that the browser already contains the minimum runtime.

A browser already has:

```
HTML  
CSS  
JavaScript  
DOM  
IDs  
data-* attributes  
Unicode  
directionality  
ShadowDOM  
SVG  
Canvas  
innerHTML  
textContent
```

That is enough to build the zero-dependency form of OMI.

No server is required.

No Node.js is required.

No WebRTC is required.

No WebSocket is required.

No MQTT is required.

No Docker is required.

No QEMU is required.

The browser can already do the essential actions:

```
render a mark  
hold an address  
store a payload  
orient text  
contain hierarchy  
project a shape  
inspect a node  
transform a view
```

So the first principle is:

```
OMI begins as notation agreement inside the browser.
```

3. The Core Browser Mapping

The browser-native OMI model resolves to this:

```
id      = high-plane address  
data-*  = low-plane payload  
bidi    = orientation switch  
DOM     = containment surface  
CSSOM   = projection surface  
JSDOM   = inspectable structure
```

```
SVG      = vector portal  
ShadowDOM = scoped enclosure  
innerHTML/textContent = linear symbolic stream
```

The minimal browser equation is:

```
id + data-* + direction + scope = addressable state
```

In OMI language:

```
x = id  
y = data-*
```

The browser gives us both directly.

Example:

```
<div id="omi-high-0420" data-omi="low-state-0017"></div>
```

This already gives us:

```
x = omi-high-0420  
y = low-state-0017
```

The object is addressable.

The payload is inspectable.

The browser is the runtime.

4. The Omicron Boundary

The static frame is:

```
omi---imo
```

The deeper symbolic anchors are:

```
0x03BF = Greek small omicron  
0x039F = Greek capital omicron
```

These act as opening and closing boundaries.

They are not payload.

They define the frame.

```
0x03BF ... 0x039F
```

or:

```
omi ... imo
```

The key rule:

```
The boundary makes payload possible.  
Therefore the boundary is not payload.
```

The frame can be inverted:

```
omi → imo  
imo → omi
```

That gives OMI its bidirectional structure.

5. The Hyphen Is the Bus

The hyphen is not decoration.

It is the visible bus delimiter.

```
omi---imo
```

The gap matters.

The relation lives in the space between the sides.

A hyphenated OMI address can be read as:

```
source - bridge - object
```

or:

```
car - relation - cdr
```

or:

```
opening - payload - closing
```

So the browser-native form becomes:

```
The ID is the address.  
The hyphen is the bus.  
The DOM is the carrier.  
Agreement is the protocol.
```

6. The Quadratic Frame

The main projection equation is:

$$Q(x,y) = 60x^2 + 16xy + 4y^2$$

This is not arbitrary.

It separates the system into three domains:

```
60x2  = high-plane address / id / orientation reserve  
16xy   = bridge / portal / cross-reference  
4y2   = low-plane data / data-* / atomic delta
```

In browser terms:

```
x = id  
y = data-*
```

So:

```
60x2 = what the object is addressed as  
4y2  = what the object carries locally  
16xy  = where address and payload meet
```

This gives a clean rule:

```
id provides address.  
data-* provides payload.  
16xy provides the portal relation.
```

7. Why 60 Matters

The coefficient `60` is used because it is highly divisible.

It supports clean subdivision:

```
60 = 1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60
```

That makes it useful for timing, orientation, and layout.

One hour divides cleanly into:

```
30 minutes  
20 minutes  
15 minutes  
12 minutes  
10 minutes  
6 minutes  
5 minutes  
4 minutes  
3 minutes  
2 minutes  
1 minute
```

In OMI, 60 becomes the period reserve.

$60x^2$ = high-plane timing and orientation capacity

It gives the browser model a clean way to think about cycles, clocks, and rotations without needing floating-point authority.

8. The Atomic Delta

OMI also needs a local bitwise motion.

That is the Atomic Delta:

$\Delta C(x) = \text{rotr}(x,1) \text{ XOR } \text{rotr}(x,3) \text{ XOR } \text{rotr}(x,2) \text{ XOR } C$

This has four design choices:

rotations
XOR
constant
mask to width

Why rotations?

No bits are lost.

Why XOR?

It is reversible.

Why a constant?

It breaks the zero fixed point.

Why mask to width?

It keeps the state bounded.

In the browser model, Atomic Delta belongs on the low side:

$4y^2 = \text{atomic local data}$

So:

Atomic Delta rolls the data-* side.
The id side gives address.
The bridge samples their relation.

9. OMI Zero

The zero-dependency seed is:

$0! = \text{omi} \text{---} \text{imo}$

Mathematically:

$0! = 1$

Structurally:

the null action resolves as identity

In OMI:

the smallest possible agreement is one bounded relation

That relation is:

$\text{omi} \text{---} \text{imo}$

So OMI Zero means:

```
Before every tower,  
before every portal,  
before every image,  
before every avatar,  
before every map,  
there is one bounded agreement.
```

10. The Nine Universal Edges

The OMI tower has twelve positions:

```
0  OMI---  
1  RULES  
2  FACTS  
3  CLOSURES  
4  COMBINATORS  
5  CONS  
6  JOIN  
7  COMPOSE  
8  ENCODE  
9  DECODE  
10 OMI---IMO  
11 RETURN :0
```

But the core data types are the nine middle positions:

```
RULES  
FACTS  
CLOSURES  
COMBINATORS  
CONS  
JOIN  
COMPOSE  
ENCODE  
DECODE
```

These are the universal edges.

They are the root predicates of OMI.

```
RULES constrain.  
FACTS ground.  
CLOSURES bound.  
COMBINATORS transform.  
CONS pair.  
JOIN binds.  
COMPOSE orders.  
ENCODE lowers.  
DECODE raises.
```

Everything else is boundary, bridge, return, or projection.

11. Forms Before Barcodes

At one point, barcodes looked like the natural physical layer.

But the deeper realization is:

```
barcodes are carriers  
not the base form
```

The base form is:

```
Omi-Form
```

An Omi-Form is any integer-native shape:

```
pixel  
voxel  
polyform  
glyph cell  
font cell  
emoji component  
SVG shape  
DOM element  
Karnaugh cell  
portal tile
```

No floating point is required.

An Omi-Form is a discrete structure that can later be rendered, hashed, encoded, or scanned.

```
Omi-Form builds.  
Omi-Glyph renders.  
Omi-Hash compiles.  
Omi-Map adjudicates.  
Omi-Image remembers.
```

12. Omi-Glyph

An Omi-Glyph is the visible sign of an Omi-Form.

It may be rendered as:

```
Unicode  
emoji  
font glyph  
SVG  
Canvas  
polyform  
voxel projection  
DOM component
```

But the glyph is not authority.

It is projection.

```
The glyph shows the form.  
The hash remembers the form.  
The frame validates the form.
```

13. Omi-Hash

An Omi-Hash is the reproducible digest of the form/glyph/frame relation.

It answers:

```
What exact structure was rendered?  
What features were active?  
What scope was used?
```

What address was declared?
What payload was bound?

The Omi-Hash can then compile a new OMI instruction:

```
Omi-Form
  ↓
Omi-Glyph
  ↓
Omi-Hash
  ↓
omi---imo long-word instruction
```

This is why OMI can be meta-circular.

The object can describe how to reconstruct itself.

14. Omi-Map

Omi-Map is the feature mapping surface.

The best mental model is a Karnaugh map.

A Karnaugh map lays Boolean states out so neighboring cells differ by one bit.

OMI uses that idea to test one feature at a time.

```
feature enters
scope receives
portal tests it
glyph changes
hash recomputes
agreement accepts or rejects
```

In short:

Omi-Map is where a single feature tries itself on inside a portal.

The feature can be treated like a state vector:

```
|feature>
```

The portal tests it with XOR:

```
portal_state XOR feature_axis
```

This does not overwrite the object.

It tests fit.

15. Omi-Portal

An Omi-Portal is the flat scoped browser surface where features are tested.

It can use custom elements:

```
<omi-fs></omi-fs>
<omi-gs></omi-gs>
<omi-rs></omi-rs>
<omi-us></omi-us>

<imo-fs></imo-fs>
<imo-gs></imo-gs>
<imo-rs></imo-rs>
<imo-us></imo-us>
```

The portal can also use:

```
ShadowDOM
SVG
innerHTML
textContent
Canvas
CSS variables
```

The difference is:

DOM nodes carry hierarchy.
textContent carries linear symbolic control.
innerHTML/SVG/ShadowDOM can act as wormhole projection surfaces.

Canonical rule:

Projection is not authority.

16. Omi-Image

Omi-Image is the bounded package that can restore the state.

It combines:

Omi-Zero
Omi-Form
Omi-Glyph
Omi-Hash
Omi-Map
Omi-Matrix
Omi-Avatar
Omi-Mirror

But at the browser-first level, it means:

a recoverable browser-native package of agreement

An Omi-Image can be:

rendered
hashed
decoded
replayed
projected
stored
restored

The image is not the source.

It preserves the relation.

The source exceeds the image.
The image preserves the path back.

17. The Zero-Dependency Stack

The complete beginner-friendly stack is:

```
OMI Zero
  one bounded notation agreement

Omi-Form
  integer-native shape

Omi-Glyph
  visible sign

Omi-Hash
  reproducible identity

Omi-Map
  feature adjudication

Omi-Portal
  browser projection surface

Omi-Image
  recoverable package
```

And underneath all of it:

```
id = x
data-* = y
unicode-bidi = direction
DOM = hierarchy
CSSOM = projection
ShadowDOM = scope
SVG = vector portal
textContent = linear symbolic stream
```

18. What New People Should Remember

OMI is not trying to make the browser complicated.

It is trying to notice what the browser already gives us.

The browser already gives us:

```
addresses
attributes
hierarchy
direction
style
projection
events
text
shapes
scope
```

OMI adds a notation agreement:

```
omi---imo
```

Then it adds a mathematical frame:

$$Q(x,y) = 60x^2 + 16xy + 4y^2$$

Then it adds root predicate data types:

```
RULES
FACTS
CLOSURES
COMBINATORS
CONS
JOIN
COMPOSE
ENCODE
DECODE
```

Then it lets the browser become the projection surface.

That is the whole first-principles model.

19. Final Canon

OMI begins with notation, not infrastructure.

The ID is the address.
The data-* attribute is the payload.
The hyphen is the bus.
The DOM is the carrier.
Unicode-bidi is the orientation switch.

$60x^2$ is the high-plane address.
 $4y^2$ is the low-plane data.
 $16xy$ is the portal bridge.

Omi-Form builds.
Omi-Glyph renders.
Omi-Hash compiles.
Omi-Map adjudicates.
Omi-Portal projects.
Omi-Image remembers.

The nine universal edges compose the world.
The three sentinels bound, bridge, and return it.

0! = omi---imo

The beginning is one bounded agreement.

The browser is already enough.